

you need to add ExSwift as a submodule. You should drag the ExSwift sub folder with source code into the Xcode file navigator. You can use it as soon as you select add to target.

<http://dgit.in/exswift>

Oriole

Oriole enables you to add useful helper methods to Swift collections and make use of Swift 2.0's new protocol extensions. This provides more natural APIs. The methods used by Oriole extend `CollectionType` including addition of collections like `Array`, `Set`, and `Dictionary`. The extension primarily leverages the possibilities of the Swift standard library and implements functional solutions to problems. Oriole is an evolving plugin and the developer is planning to infuse more features.

<http://dgit.in/oriole-ext>

OptionalExtensions

`OptionalExtensions` provides useful utility methods (also called operators) which will help you to implement complex functionalities easily. The operators included with `OptionalExtensions` are `filter`, `mapNil`, `flatMapNil`, `then`, `maybe`, `onSome`, `onNone`, `isSome` and `isNone`. To work with the extension, you need to add `optionalExtensions.swift` to your project. Alternatively, you can install via CocoaPods and Carthage

<http://dgit.in/optional-ext>

SwiftGestureRecognition

`SwiftGestureRecognition` enables you to extend `UIGestureRecognizer`s to help you to prototype with them in Xcode Playgrounds. To work with the extension, you need to drag and drop `SwiftGestureRecognition.xcodeproj` into the workspace. After that you need to import it into your playground by using the following code

- `import SwiftGestureRecognition`

To use the library, you need to call `UIPanGestureRecognizer()` method.

<http://dgit.in/swift-gesture>

Colors

`Colors` is a pure Swift library to work in relation with ANSI codes and it simplifies your command-line coloring and styling. To work with the extension, you need to have access to Xcode 7.3 and Swift 2.2. Moreover,